



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

Diseño e Implementación de una Plataforma Analítica para el Análisis y Predicción de Patrones de Movilidad Urbana en Nueva York

TRABAJO FIN DE MÁSTER

Máster en Big Data Analytics

Autor: Carlos Monteagudo Tobarra

Tutor: Fernando Martínez Plumed

Curso 2024–2025

Resum

Aquest Treball de Fi de Màster presenta el disseny i la implementació d'una plataforma analítica reproduïble per a l'estudi dels patrons de mobilitat urbana a la ciutat de Nova York i la predicció de la demanda de serveis de taxi i vehicles de lloguer amb conductor. El treball utilitza com a font principal els registres de viatges publicats per la New York City Taxi and Limousine Commission (TLC) i considera les diferents modalitats disponibles: Yellow Taxi, Green Taxi, For-Hire Vehicle (FHV) i High Volume For-Hire Vehicle (HVFHV). L'horitzó temporal d'estudi comprèn el període 2018–2025, fet que permet analitzar tant el comportament previ a la pandèmia de COVID-19 com la ruptura de mobilitat produïda durant 2020 i la posterior recuperació.

La solució s'ha concebut sota una arquitectura de medalló. La capa Bronze conserva els fitxers originals en format Parquet, mentre que les capes Silver i Gold s'implementen sobre PostgreSQL i es transformen mitjançant dbt. La ingesta, la planificació i el control operacional del flux es centralitzen en Apache NiFi, que organitza el processament mitjançant grups de processos, paràmetres, rutes d'èxit i error, reintents i mecanismes de procedència de dades. Docker Compose encapsula els serveis necessaris i permet desplegar de manera reproduïble NiFi, PostgreSQL, dbt, l'entorn Python de machine learning i Apache Superset. El model Gold proporciona taules de fets, dimensions i *data marts* orientats a l'anàlisi de demanda, mobilitat, rendiment econòmic i oportunitats operatives per als conductors.

La component predictiva formula la demanda com el nombre de viatges iniciats per zona TLC i franja horària. S'estableixen dos horitzons de predicció, una hora i vint-i-quatre hores, i es comparen tres famílies de models supervisats: regressió de Poisson, boscos aleatoris i gradient boosting amb LightGBM. L'avaluació segueix una partició estrictament temporal per evitar fuites d'informació i utilitza mètriques com MAE, RMSE i WAPE. Els resultats experimentals definitius s'incorporaran a partir de l'execució reproduïble de la versió final del repositori.

Paraules clau: mobilitat urbana, taxi, NYC TLC, Apache NiFi, arquitectura de medalló, enginyeria de dades, PostgreSQL, dbt, Docker, Apache Superset, predicció de demanda.

Resumen

El presente Trabajo Fin de Máster describe el diseño y la implementación de una plataforma analítica reproducible para estudiar los patrones de movilidad urbana en la ciudad de Nueva York y predecir la demanda de servicios de taxi y vehículos de alquiler con conductor. La fuente principal son los registros de viajes publicados por la New York City Taxi and Limousine Commission (TLC), integrando las modalidades Yellow Taxi, Green Taxi, For-Hire Vehicle (FHV) y High Volume For-Hire Vehicle (HVFHV). El periodo de estudio comprende 2018–2025, de modo que el análisis permite comparar la movilidad previa a la pandemia de COVID-19, la disrupción observada durante 2020 y el proceso de recuperación posterior.

La plataforma se estructura siguiendo una arquitectura de medallón. La capa Bronze conserva los ficheros de origen en formato Parquet; las capas Silver y Gold se materializan en PostgreSQL y se transforman mediante dbt. La ingesta, planificación y control operacional del flujo se centralizan en Apache NiFi, que organiza el procesamiento mediante grupos de procesos, parámetros, rutas de éxito y error, reintentos y mecanismos de procedencia del dato. Docker Compose encapsula los servicios necesarios y permite desplegar de forma reproducible NiFi, PostgreSQL, dbt, el entorno Python de machine learning y Apache Superset. La capa Gold contiene dimensiones, tablas de hechos y *data marts* diseñados para responder a preguntas de negocio relacionadas con demanda, movilidad, congestión aproximada, rendimiento económico y oportunidades operativas para conductores.

La predicción se formula como un problema de regresión sobre el número de viajes iniciados por zona TLC y hora. Se consideran horizontes de una hora y veinticuatro horas y se plantea la comparación de tres aproximaciones complementarias: regresión de Poisson, Random Forest y LightGBM. La evaluación utiliza particiones temporales para evitar fuga de información y métricas MAE, RMSE y WAPE. La memoria separa explícitamente los resultados observados de las estimaciones y simulaciones de rentabilidad, y reserva las cifras experimentales definitivas para la ejecución reproducible del repositorio final.

Palabras clave: movilidad urbana, taxi, NYC TLC, Apache NiFi, arquitectura medallón, ingeniería de datos, PostgreSQL, dbt, Docker, Apache Superset, predicción de demanda.

Abstract

This Master’s Thesis presents the design and implementation of a reproducible analytical platform for studying urban mobility patterns in New York City and forecasting demand for taxi and for-hire vehicle services. The primary source is the trip-record data published by the New York City Taxi and Limousine Commission (TLC), integrating Yellow Taxi, Green Taxi, For-Hire Vehicle (FHV), and High Volume For-Hire Vehicle (HVFHV) services. The analysis covers the 2018–2025 period, enabling a longitudinal comparison between pre-pandemic mobility, the disruption associated with COVID-19 in 2020, and the subsequent recovery.

The solution follows a Medallion Architecture. The Bronze layer preserves the original Parquet files, whereas the Silver and Gold layers are implemented in PostgreSQL and transformed with dbt. Data ingestion, scheduling and operational flow control are centralised in Apache NiFi, using process groups, parameters, explicit success and failure routes, retry policies and data-provenance capabilities. Docker Compose packages the required services and provides a reproducible deployment for NiFi, PostgreSQL, dbt, the Python machine-learning environment and Apache Superset. The Gold layer exposes dimensions, fact tables and analytical data marts intended to support demand analysis, mobility patterns, approximate congestion indicators, economic performance and driver-opportunity analysis.

Demand forecasting is formulated as a regression problem over the number of trips originating in each TLC zone and hourly time slot. Two forecasting horizons are considered: one hour and twenty-four hours. Three complementary supervised models are compared: Poisson regression, Random Forest and LightGBM. Evaluation is based on strictly temporal splits to avoid information leakage, using MAE, RMSE and WAPE. Final experimental figures are intentionally tied to the reproducible execution of the final repository rather than being estimated or manually introduced.

Key words: urban mobility, taxi, NYC TLC, Apache NiFi, Medallion Architecture, data engineering, PostgreSQL, dbt, Docker, Apache Superset, demand forecasting.

Siglas y abreviaturas

BI	Business Intelligence
COVID-19	Coronavirus Disease 2019
DAG	Directed Acyclic Graph, grafo dirigido acíclico
DBT	data build tool
DWH	Data Warehouse
ELT	Extract, Load, Transform
ETL	Extract, Transform, Load
FHV	For-Hire Vehicle
HVHV	High Volume For-Hire Vehicle
KPI	Key Performance Indicator
MAE	Mean Absolute Error
ML	Machine Learning
NYC	New York City
OD	Origen-Destino
PCA	Principal Component Analysis
RMSE	Root Mean Squared Error
SQL	Structured Query Language
TLC	New York City Taxi and Limousine Commission
WAPE	Weighted Absolute Percentage Error

Índice general

Índice general	VII
Índice de figuras	XI
Índice de tablas	XI
1 Introducción	1
1.1 Motivación	1
1.2 Problema de investigación	2
1.3 Objetivo general	2
1.4 Objetivos específicos	2
1.5 Alcance y exclusiones	3
1.6 Metodología de trabajo	4
1.7 Contribuciones principales	4
1.8 Estructura de la memoria	4
2 Movilidad urbana y contexto del problema	7
2.1 La movilidad urbana como sistema de datos	7
2.2 La New York City Taxi and Limousine Commission	7
2.3 Yellow Taxi	8
2.4 Green Taxi	8
2.5 For-Hire Vehicles	8
2.6 High Volume For-Hire Vehicles	9
2.7 Taxi Zones y representación espacial	9
2.8 Dimensión temporal y estacionalidad	9
2.9 Impacto de la pandemia de COVID-19	10
2.10 Meteorología y factores externos	10
2.11 Preguntas de negocio	10
2.11.1 Evolución de la movilidad	10
2.11.2 Distribución geográfica	11
2.11.3 Rendimiento operativo y económico	11
2.11.4 Predicción de demanda	11
2.12 Del análisis descriptivo al producto analítico	11
3 Estado del arte	13
3.1 De ETL a plataformas analíticas reproducibles	13
3.2 Data Warehouse, Data Lake y Lakehouse	13
3.3 Arquitectura de Medallón	14
3.4 Modelado dimensional	14
3.5 DataOps, versionado y reproducibilidad	14
3.6 Plataformas de flujo de datos y Apache NiFi	15
3.7 Calidad de datos	15
3.8 Datos de taxi como fuente para estudiar movilidad	16
3.9 Predicción de demanda de taxi	16
3.10 Regresión de Poisson como referencia estructurada	16
3.11 Random Forest	17

3.12	Gradient Boosting y LightGBM	17
3.13	Validación temporal y fuga de información	17
3.14	Métricas de evaluación	17
3.15	Posicionamiento del presente trabajo	18
4	Marco tecnológico	19
4.1	Criterios de selección	19
4.2	Docker y Docker Compose	19
4.3	Apache NiFi	20
4.4	NiFi y PostgreSQL	20
4.5	Apache Parquet	20
4.6	PostgreSQL	21
4.7	dbt	21
4.8	Python	21
4.9	scikit-learn	21
4.10	LightGBM	22
4.11	Apache Superset	22
4.12	Redis	22
4.13	Git, GitHub y versionado de flujos	22
4.14	Relación entre componentes	22
4.15	Tecnologías deliberadamente no incorporadas	23
5	Fuentes de datos y análisis preliminar	25
5.1	Fuente principal: TLC Trip Record Data	25
5.2	Periodo de estudio	25
5.3	Disponibilidad por servicio	25
5.4	Unidad de ingesta	26
5.5	Cambios de esquema	26
5.6	Taxonomía común del viaje	27
5.7	Taxi Zone Lookup y geometrías	27
5.8	Calendario y festivos	27
5.9	Meteorología	28
5.10	Reglas iniciales de calidad	28
5.11	Outliers y valores extremos	29
5.12	Análisis exploratorio previsto	29
5.12.1	Volumen y cobertura	29
5.12.2	Patrones temporales	29
5.12.3	Patrones geográficos	29
5.12.4	Indicadores económicos	30
5.13	Trazabilidad de la fuente al indicador	30
6	Diseño de la arquitectura	31
6.1	Requisitos de arquitectura	31
6.2	Evolución de la arquitectura	31
6.3	Comparación de alternativas de ingesta	31
6.4	Visión general	32
6.5	Capa Bronze	32
6.6	Capa Silver	33
6.7	Capa Gold	33
6.8	Modelo dimensional	34
6.9	NiFi como capa de ingesta y automatización	34
6.10	Parameter Contexts	34
6.11	Esquema de control	35
6.12	Idempotencia	35

6.13	Data Provenance y observabilidad	35
6.14	Reintentos y tratamiento de errores	36
6.15	Versionado de los flujos	36
6.16	Integración con dbt y machine learning	36
6.17	Seguridad y secretos	36
6.18	Modo demo y modo full	37
6.19	Reproducibilidad	37
7	Implementación del pipeline de datos	39
7.1	Organización del repositorio	39
7.2	Inicialización del entorno	40
7.3	Configuración de NiFi	40
7.4	Descubrimiento de ficheros TLC	40
7.5	Descarga mediante InvokeHTTP	41
7.6	Validación y publicación en Bronze	41
7.7	Rutas de éxito, fallo y cuarentena	41
7.8	Conexión con PostgreSQL	42
7.9	Carga hacia el área relacional	42
7.10	Construcción de Silver con dbt	42
7.11	Tests de dbt y promoción	43
7.12	Construcción de Gold	43
7.13	Fuentes externas	43
7.14	Control de ejecuciones	43
7.15	Data Provenance	44
7.16	Planificación	44
7.17	Activación de machine learning	44
7.18	Modo demo	44
7.19	Modo full	45
7.20	Pruebas automatizadas	45
7.21	Rendimiento y uso de recursos	45
7.22	Documentación operativa	46
	Bibliografía	47

Índice de figuras

4.1	Relación funcional entre las principales tecnologías del proyecto con Apache NiFi como capa de ingesta y automatización.	23
5.1	Evolución mensual del volumen de viajes por tipo de servicio.	29
6.1	Arquitectura lógica end-to-end con Apache NiFi como capa de ingesta y automatización.	32
6.2	Esquema estrella simplificado para la demanda horaria por zona.	34
6.3	Organización conceptual de los grupos de procesos de Apache NiFi.	34

Índice de tablas

5.1	Servicios TLC incluidos y cobertura analítica prevista.	26
5.2	Contrato conceptual común para un viaje Silver.	27
6.1	Comparación entre la implementación inicial y la arquitectura NiFi.	32

CAPÍTULO 1

Introducción

La disponibilidad de datos masivos de movilidad ha modificado de forma sustancial la manera en que pueden estudiarse los sistemas de transporte urbano. Frente a los enfoques basados exclusivamente en encuestas, conteos manuales o muestras reducidas, los registros transaccionales generados por servicios de taxi y vehículos de alquiler con conductor permiten observar millones de desplazamientos con una granularidad temporal y espacial elevada. Nueva York constituye un caso especialmente relevante por la dimensión de su mercado de transporte bajo demanda y por la política de datos abiertos desarrollada por la New York City Taxi and Limousine Commission (TLC), que publica registros de viajes de distintas modalidades de servicio [29, 30].

El valor de estos datos no reside únicamente en su volumen. La combinación de marcas temporales, zonas de origen y destino, duración, distancia, tarifas y otros atributos hace posible analizar regularidades de movilidad, estudiar diferencias entre servicios, detectar episodios anómalos y construir modelos predictivos de demanda. Sin embargo, trabajar con un histórico de varios años y con múltiples tipos de servicio plantea un problema de ingeniería de datos antes incluso de abordar el análisis estadístico. Los esquemas evolucionan, algunos campos aparecen o desaparecen, existen registros incompletos o inconsistentes y la escala obliga a separar el almacenamiento bruto de los modelos analíticos orientados a consulta.

Este proyecto aborda conjuntamente ambas dimensiones. Por una parte, se diseña una plataforma de datos local, reproducible y contenerizada, capaz de descargar, validar, transformar y modelar los registros de la comisión; por otra, se construye una capa analítica destinada a describir los patrones de movilidad y a predecir la demanda horaria por zona y tipo de servicio. El resultado se concibe como un proyecto *end-to-end*: desde la obtención del fichero de origen hasta la visualización de indicadores y predicciones en una herramienta de Business Intelligence.

1.1 Motivación

La motivación del trabajo se apoya en tres argumentos complementarios. El primero es de naturaleza urbana y económica. La demanda de taxi no se distribuye de manera homogénea. Cambia según la hora, el día de la semana, la localización, la climatología, el calendario y la presencia de acontecimientos extraordinarios. Estudios previos han mostrado que la demanda de taxi en Nueva York presenta fuertes dependencias espacio-temporales y que la inclusión de estructura espacial y temporal mejora la capacidad predictiva [34, 16, 37]. Desde el punto de vista operativo, anticipar estas variaciones puede contribuir a posicionar mejor la oferta, reducir tiempos improductivos y comprender cómo compiten o se complementan los diferentes servicios.

El segundo argumento es metodológico. Los conjuntos de datos de movilidad suelen utilizarse en ejercicios de análisis o modelado en los que la preparación previa queda reducida a un conjunto de scripts puntuales. Esa aproximación resulta suficiente para una prueba de concepto, pero no reproduce las prácticas habituales de una plataforma de datos mantenible. En un escenario real son necesarios controles de estado, idempotencia, registros de ejecución, validaciones, separación de capas, versionado de transformaciones y mecanismos de despliegue reproducible. El presente trabajo utiliza el caso de movilidad como vehículo para demostrar estas prácticas de ingeniería de datos.

El tercer argumento es académico. La combinación de arquitectura, modelado dimensional, análisis de negocio, calidad de datos y aprendizaje automático permite estudiar el ciclo completo de creación de un producto analítico. La arquitectura de medallón proporciona una separación conceptual entre datos de origen, información depurada y modelos de negocio [13]; dbt permite tratar las transformaciones SQL como código versionado y testeable [14]; Docker facilita que la solución sea ejecutable con las mismas dependencias en distintos equipos [15]; y Apache Superset proporciona una capa de consumo interactiva sobre los modelos analíticos [9].

1.2 Problema de investigación

El problema central puede formularse de la siguiente manera: *¿cómo diseñar una plataforma reproducible que permita transformar un histórico masivo y heterogéneo de viajes de la TLC en información analítica fiable, y hasta qué punto los patrones históricos permiten predecir la demanda futura por zona y franja horaria?*

Esta pregunta combina dos problemas relacionados. El primero es arquitectónico: definir un flujo capaz de incorporar diferentes familias de datos, evolucionar ante cambios de esquema, evitar reprocesamientos innecesarios y dejar trazabilidad de cada ejecución. El segundo es predictivo: construir un conjunto de variables que represente adecuadamente la periodicidad de la demanda y evaluar modelos con una metodología temporal que evite utilizar información del futuro durante el entrenamiento.

La plataforma se diseña para ejecutarse en un único equipo con Docker Desktop, pero aplicando principios que puedan trasladarse a un entorno servidor. Esta restricción es deliberada. El objetivo no es construir una infraestructura distribuida a cualquier coste, sino demostrar que una arquitectura rigurosa puede implementarse con tecnologías abiertas y recursos locales cuando el diseño separa adecuadamente almacenamiento, transformación, control y consumo.

1.3 Objetivo general

El objetivo general del Trabajo Fin de Máster es **diseñar e implementar una plataforma analítica reproducible para analizar los patrones de movilidad de los servicios de taxi y vehículos de alquiler en Nueva York y predecir la demanda de viajes por zona y franja temporal.**

1.4 Objetivos específicos

A partir del objetivo general se definen los siguientes objetivos específicos:

1. Diseñar una arquitectura de datos contenerizada que pueda levantarse mediante Docker Compose y que no requiera instalaciones locales de PostgreSQL, dbt o Apache Superset.
2. Automatizar el descubrimiento y la descarga incremental de los ficheros publicados por la TLC para las modalidades Yellow Taxi, Green Taxi, FHV y HVFHV.
3. Preservar los datos de origen en una capa Bronze basada en Parquet, manteniendo metadatos de procedencia, fecha, servicio y estado de procesamiento.
4. Normalizar y depurar la información en una capa Silver, armonizando esquemas y aplicando reglas explícitas de calidad.
5. Diseñar una capa Gold basada en modelado dimensional y *data marts* orientados a movilidad, comparación de servicios, rendimiento económico y predicción.
6. Incorporar mecanismos de idempotencia, auditoría, control de errores, cuarentena y observabilidad para que el pipeline pueda reejecutarse sin duplicar o corromper resultados.
7. Enriquecer los viajes con dimensiones temporales, información geográfica, festivos y variables meteorológicas cuando su disponibilidad y granularidad lo permitan.
8. Analizar la evolución de la movilidad entre 2018 y 2025, con especial atención al cambio estructural asociado a la pandemia de COVID-19 y a la evolución posterior.
9. Formular la predicción como el número de viajes iniciados por zona TLC y hora, evaluando horizontes de una hora y veinticuatro horas.
10. Comparar tres familias de modelos supervisados con propiedades diferentes: regresión de Poisson, Random Forest y LightGBM, además de un baseline temporal de referencia.
11. Construir dashboards en Apache Superset que permitan explorar resultados descriptivos, geográficos, económicos y predictivos.
12. Documentar la solución de forma que una tercera persona pueda reproducir el modo de demostración a partir del repositorio público.

1.5 Alcance y exclusiones

El trabajo analiza el periodo 2018–2025. Esta ventana proporciona años inmediatamente anteriores a la pandemia, el periodo de mayor disrupción y varios años posteriores. No obstante, la cobertura efectiva varía según la familia de servicio. En particular, la TLC comenzó a publicar los registros HVFHV como conjunto diferenciado en 2019 [29]; por tanto, no se fuerza una homogeneidad temporal inexistente y cada análisis declara el periodo realmente disponible.

El concepto de rentabilidad se aborda mediante indicadores observados y mediante escenarios parametrizables. Los registros TLC contienen importes asociados a determinados servicios, pero no representan una contabilidad completa del conductor. Costes como combustible, mantenimiento, seguros, amortización, licencias o tiempo sin pasajero no pueden inferirse de forma fiable para todos los registros. En consecuencia, el trabajo distingue entre *ingreso observado*, *indicador derivado* y *beneficio estimado*. La memoria evita presentar como beneficio neto una magnitud que no se observa directamente.

Tampoco se plantea un sistema de predicción en tiempo real con garantías de servicio. El objetivo es construir y evaluar un pipeline reproducible de entrenamiento e inferencia por lotes. Una evolución hacia inferencia *streaming*, alta disponibilidad o despliegue en nube se considera una línea futura.

1.6 Metodología de trabajo

El desarrollo sigue una aproximación incremental. En primer lugar se inventaría y audita el código existente, identificando componentes reutilizables y riesgos de publicación. A continuación se estabiliza el entorno Docker y la capa de control. Sobre esa base se implementan las fases de ingesta y transformación. La capa analítica se diseña una vez fijada la granularidad de las tablas Silver, y el modelado predictivo consume exclusivamente entidades Gold preparadas para ese propósito.

Desde el punto de vista experimental, las decisiones se separan en dos grupos. Las decisiones arquitectónicas se validan mediante reproducibilidad, tests, reejecución e inspección de estado. Las decisiones de modelado se validan mediante métricas fuera de muestra y particiones temporales. Esta separación es importante: que un modelo obtenga una buena métrica no compensa un pipeline no reproducible, y una arquitectura técnicamente correcta no implica que las predicciones sean útiles.

El control de versiones con Git registra el desarrollo y permite asociar cada experimento a una revisión concreta del código. Los resultados finales deberán referenciar el *commit* o etiqueta utilizado para que la comparación de modelos pueda repetirse. El repositorio definitivo se publicará en <https://github.com/Cyos98/TFM>.

1.7 Contribuciones principales

Las principales contribuciones esperadas son las siguientes:

- una arquitectura local de datos reproducible que integra almacenamiento de ficheros, PostgreSQL, dbt, validación, machine learning y Superset;
- un proceso de ingestión incremental e idempotente para varios años y varios tipos de servicio regulados por la comisión;
- un modelo dimensional común que permite comparar modalidades con esquemas diferentes sin ocultar sus particularidades;
- un conjunto de KPIs y *data marts* orientados a comprender patrones de movilidad y oportunidades operativas;
- una evaluación predictiva homogénea de tres familias de modelos para dos horizontes temporales y varios servicios;
- un entregable reproducible que puede ejecutarse en modo reducido para demostración y en modo completo cuando existen recursos suficientes.

1.8 Estructura de la memoria

El resto de la memoria comienza con el capítulo 2, que introduce el contexto de movilidad urbana y describe los servicios regulados por la TLC. Los capítulos posteriores

desarrollarán el estado del arte, las tecnologías y fuentes de datos seleccionadas, la arquitectura y la implementación del pipeline, la capa de inteligencia de negocio, la metodología predictiva, los resultados experimentales y, finalmente, las conclusiones, limitaciones y líneas futuras.

CAPÍTULO 2

Movilidad urbana y contexto del problema

2.1 La movilidad urbana como sistema de datos

La movilidad urbana es el resultado agregado de decisiones individuales condicionadas por la estructura de la ciudad, la oferta de transporte, el calendario laboral, la actividad económica y numerosos factores externos. En una gran metrópolis, estas decisiones generan patrones recurrentes —por ejemplo, picos asociados a desplazamientos laborales—, pero también variaciones locales y episodios extraordinarios. Analizar movilidad implica, por tanto, combinar una dimensión temporal con una dimensión espacial.

Los servicios de taxi y transporte bajo demanda constituyen una fuente especialmente informativa porque cada viaje representa una interacción efectiva entre una demanda y una oferta de movilidad. El origen indica dónde se materializa la necesidad de desplazamiento; el destino aproxima la dirección del flujo; la hora revela periodicidades; y la duración, la distancia y el importe permiten estudiar características económicas y operativas. Cuando se dispone de millones de observaciones, la red de viajes puede analizarse como una estructura de flujos urbanos. Riascos y Mateos estudiaron más de mil millones de viajes de taxi en Nueva York y mostraron que las matrices origen-destino permiten caracterizar lugares especialmente relevantes dentro de la red urbana [33].

Esta riqueza presenta una contrapartida: los registros de viaje no son una observación completa de la movilidad de la ciudad. Representan únicamente los servicios regulados por la TLC y reflejan las reglas, tecnologías y prácticas de captura de cada periodo. Por ello, las inferencias deben limitarse al dominio de los datos y evitar identificar automáticamente *demanda de taxi* con *movilidad total*.

2.2 La New York City Taxi and Limousine Commission

La TLC es el organismo de la ciudad de Nueva York responsable de regular diferentes modalidades de transporte de pasajeros. Además de su función regulatoria, publica conjuntos de datos de viajes con fines de transparencia, investigación y análisis. La documentación oficial indica que los registros mensuales se ponen a disposición en formato Parquet y se acompañan de diccionarios de datos específicos para Yellow Taxi, Green Taxi, FHV y HVFHV [29, 30].

Los datos se distribuyen como ficheros mensuales, circunstancia que influye directamente en el diseño del pipeline. El fichero mensual constituye una unidad natural de

descubrimiento, descarga, validación y estado. En lugar de interpretar la fuente como un único dataset estático, el proyecto la modela como una secuencia temporal de activos cuya disponibilidad y esquema pueden evolucionar.

2.3 Yellow Taxi

El taxi amarillo o *Yellow Taxi* es la modalidad más reconocible del sistema neoyorquino. Históricamente concentra una parte muy relevante de los viajes en Manhattan y dispone de un conjunto de atributos especialmente rico para el análisis económico: marcas temporales de recogida y bajada, zonas, distancia, método de pago y componentes de tarifa, entre otros. Esta riqueza lo convierte en el servicio más adecuado para validar de extremo a extremo indicadores como ingreso por viaje, ingreso por milla o distribución de propinas.

Los Yellow Taxi también han sido ampliamente estudiados en la literatura. Hochmair utilizó decenas de millones de registros para analizar variaciones espacio-temporales, infraestructuras de transporte y velocidad de viaje [20]. Safikhani et al. formularon la demanda como un proceso espacio-temporal y evaluaron modelos autorregresivos con regularización sobre datos de taxi amarillo de Nueva York [34]. A partir de estos trabajos, el TFM adopta dos decisiones: utilizar la zona y la hora como unidades fundamentales y considerar la dependencia temporal como una fuente principal de capacidad predictiva.

2.4 Green Taxi

Los Green Taxi o *Street Hail Livery* fueron concebidos para ampliar el servicio de recogida en zonas tradicionalmente menos cubiertas por los taxis amarillos. Desde la perspectiva analítica, permiten comparar patrones geográficos y económicos con Yellow Taxi utilizando un esquema parcialmente similar, aunque no idéntico.

El proyecto evita forzar ambos servicios a compartir todas las columnas. En Silver se define un núcleo semántico común —tipo de servicio, fecha de recogida, fecha de bajada, zonas, duración y campos económicos homologables—, mientras que los atributos específicos permanecen disponibles en modelos auxiliares cuando aportan información. De esta manera, la comparabilidad no exige eliminar las particularidades de cada fuente.

2.5 For-Hire Vehicles

La categoría FHV agrupa servicios de vehículos de alquiler con conductor gestionados mediante bases de despacho. La información publicada no contiene necesariamente el mismo nivel tarifario que los taxis tradicionales y su esquema ha evolucionado con las obligaciones regulatorias. Por ello, su principal utilidad dentro de este trabajo se centra en la movilidad y la demanda, no en una comparación económica campo a campo.

Esta asimetría ejemplifica un principio general del proyecto: un modelo común debe basarse en significados equivalentes y no en nombres de columna aparentemente parecidos. Las transformaciones de Silver incorporan una matriz de correspondencia de campos y reglas por servicio y periodo, de forma que cada atributo compartido tenga una definición explícita.

2.6 High Volume For-Hire Vehicles

Los HVFHV representan servicios de alto volumen asociados al crecimiento de plataformas de movilidad bajo demanda. La TLC publica desde 2019 un conjunto separado y más detallado para esta categoría [29]. Esta diferencia temporal implica que el análisis 2018–2025 no puede presentar una serie HVFHV completa desde 2018. El periodo previo sirve para Yellow, Green y FHV, mientras que las comparaciones que involucren HVFHV comienzan cuando existe cobertura específica.

La incorporación de HVFHV es relevante porque la estructura del mercado de movilidad de Nueva York ha cambiado con la expansión de las plataformas de *ride-hailing*. Trabajos previos han observado diferencias espaciales y temporales entre taxis tradicionales y servicios basados en aplicaciones, y han señalado la necesidad de considerar simultáneamente dependencias espaciales y temporales [16, 37].

2.7 Taxi Zones y representación espacial

La TLC utiliza un sistema de zonas para representar orígenes y destinos. La guía oficial proporciona tanto una tabla de correspondencias como geometrías de las zonas, aproximadamente basadas en áreas vecinales de la ciudad [30]. Esta representación es especialmente adecuada para el presente trabajo por tres motivos.

En primer lugar, reduce la complejidad asociada a coordenadas GPS individuales y proporciona una taxonomía estable para agregación. En segundo lugar, puede conectarse directamente con los identificadores de zona incluidos en los ficheros modernos. En tercer lugar, facilita la construcción de visualizaciones geográficas y rankings comprensibles para un usuario de negocio.

La unidad espacial principal del modelo predictivo es la zona de recogida. Para cada combinación de servicio, zona y hora se calcula el número de viajes iniciados. Las zonas sin viajes deben representarse explícitamente con demanda cero cuando formen parte del universo analizado; de lo contrario, el dataset de entrenamiento estaría sesgado hacia observaciones positivas.

2.8 Dimensión temporal y estacionalidad

La demanda de transporte presenta ciclos de distinta escala. La hora del día captura patrones de entrada y salida laboral, ocio nocturno y movilidad hacia aeropuertos. El día de la semana distingue jornadas laborales de fines de semana. El mes y la estación recogen efectos de clima y actividad turística. Los festivos introducen excepciones recurrentes que no quedan bien descritas únicamente con el número de día.

La dimensión temporal de Gold se diseña para representar estas propiedades de manera explícita. Además de atributos categóricos, el modelado puede incorporar codificaciones cíclicas. Para una variable periódica de periodo P , como la hora del día, se definen:

$$x_{\sin} = \sin\left(2\pi\frac{x}{P}\right), \quad x_{\cos} = \cos\left(2\pi\frac{x}{P}\right). \quad (2.1)$$

Esta representación evita que la distancia numérica entre las 23:00 y las 00:00 sea artificialmente elevada y permite a modelos lineales capturar parte de la periodicidad.

2.9 Impacto de la pandemia de COVID-19

La selección de 2018 como inicio del periodo responde principalmente a la posibilidad de disponer de una base pre-pandemia suficientemente amplia. La pandemia provocó cambios extraordinarios en la movilidad urbana desde 2020. En el caso de Nueva York, investigaciones recientes han analizado específicamente el impacto espacio-temporal de COVID-19 sobre la demanda de taxi, encontrando que sus efectos no fueron homogéneos entre zonas y que variables urbanas y epidemiológicas contribuyeron a explicar la variación [39].

En este TFM el efecto de la pandemia se estudia desde una perspectiva descriptiva y temporal. No se pretende identificar causalidad entre políticas concretas y cambios de demanda. Se comparan niveles, distribución geográfica y ritmos de recuperación entre servicios, utilizando periodos predefinidos y evitando atribuir a COVID-19 cualquier variación que pueda estar asociada a otros factores.

Una consecuencia importante para el machine learning es la presencia de un cambio de régimen. Un modelo entrenado únicamente con años previos a 2020 puede fallar de forma extrema durante la disrupción; un modelo entrenado con todo el periodo puede aprender comportamientos que mezclan contextos heterogéneos. Por ello, el capítulo experimental deberá reportar explícitamente la composición temporal de entrenamiento, validación y test y, cuando sea útil, resultados segmentados por periodos.

2.10 Meteorología y factores externos

La demanda de taxi puede estar relacionada con condiciones meteorológicas, aunque el efecto no es necesariamente lineal ni idéntico en todos los servicios. La literatura sobre predicción de demanda de taxi considera habitualmente variables externas como meteorología, calendario o uso del suelo [34, 26]. El proyecto incorpora una fuente meteorológica oficial de NOAA/NCEI con granularidad compatible con la agregación temporal, siempre que el pipeline final garantice su disponibilidad y trazabilidad [27].

Entre las variables candidatas se incluyen temperatura, precipitación, nieve, viento y visibilidad. El objetivo no es crear un modelo meteorológico, sino representar condiciones que puedan modificar la preferencia relativa por un servicio de transporte. Las variables se agregan a escala horaria o diaria según la fuente y se documenta el método de alineación temporal.

2.11 Preguntas de negocio

La capa analítica se diseña para responder cuatro grupos de preguntas.

2.11.1. Evolución de la movilidad

¿Qué volumen de viajes se realiza por servicio, año, mes, día y hora? ¿Cómo cambia a lo largo del tiempo la participación relativa de los cuatro servicios analizados? ¿Qué diferencias aparecen antes, durante y después de 2020?

2.11.2. Distribución geográfica

¿Qué zonas concentran más recogidas y destinos? ¿Existen zonas cuyo patrón depende especialmente de una franja temporal? ¿Cómo cambia la distribución entre boroughs y servicios?

2.11.3. Rendimiento operativo y económico

Para los servicios con información de tarifa suficiente, ¿qué zonas y franjas presentan mayores ingresos observados por viaje, milla u hora ocupada? ¿Qué combinaciones ofrecen una relación favorable entre demanda y duración? Estas métricas no equivalen a beneficio neto, pero sirven para describir oportunidades relativas.

2.11.4. Predicción de demanda

Dado el historial disponible hasta un instante t , ¿cuántos viajes se iniciarán en cada zona una hora más tarde y veinticuatro horas más tarde? ¿Qué modelo mantiene mejor rendimiento fuera de muestra? ¿El modelo ganador es el mismo para todos los servicios?

2.12 Del análisis descriptivo al producto analítico

El propósito final no es producir únicamente gráficos estáticos, sino transformar las preguntas anteriores en activos reutilizables. Cada KPI debe estar asociado a una tabla Gold o a una definición SQL reproducible; cada dashboard debe consumir modelos documentados; y cada predicción debe persistir junto con el modelo, horizonte y fecha de generación. Esta disciplina conecta el análisis de movilidad con la ingeniería de datos y constituye uno de los elementos diferenciales del proyecto.

CAPÍTULO 3

Estado del arte

3.1 De ETL a plataformas analíticas reproducibles

Los sistemas analíticos tradicionales se han descrito históricamente mediante procesos ETL: extracción desde sistemas de origen, transformación en una plataforma intermedia y carga en un almacén de datos. La evolución de la capacidad de cómputo de los motores analíticos favoreció posteriormente estrategias ELT, donde los datos se cargan antes de aplicar las transformaciones lógicas [38]. La diferencia no es meramente terminológica. En un enfoque ELT, las transformaciones pueden declararse como SQL versionado y ejecutarse cerca del almacenamiento, lo que facilita trazabilidad, revisión de código y pruebas automatizadas.

El proyecto adopta una solución híbrida adaptada a la naturaleza de sus fuentes. La extracción y preservación de Parquet se realiza fuera de PostgreSQL, mientras que la estandarización relacional y los modelos de negocio se expresan mediante dbt sobre PostgreSQL. De este modo se evita duplicar innecesariamente los archivos brutos en un motor relacional y, al mismo tiempo, se conserva la capacidad de construir modelos SQL auditables.

3.2 Data Warehouse, Data Lake y Lakehouse

El *Data Warehouse* organiza información integrada para consulta analítica y suele asociarse a esquemas estructurados y modelos dimensionales. Inmon formalizó una visión corporativa e integrada del almacén de datos [21], mientras que Kimball desarrolló una metodología orientada a procesos de negocio y modelos dimensionales con hechos y dimensiones [23]. Ambos enfoques han influido de manera decisiva en las arquitecturas analíticas actuales.

El concepto de *Data Lake* surgió para almacenar grandes volúmenes de información en formatos diversos con menor exigencia de modelado previo. Esta flexibilidad resuelve ciertos problemas de ingestión, pero puede trasladar la complejidad hacia el consumo si no se establecen contratos, metadatos y procesos de refinamiento. Las arquitecturas *Lakehouse* buscan combinar almacenamiento flexible con capacidades de gestión y analítica estructurada.

En un entorno local, el presente proyecto no pretende replicar todas las capacidades de un Lakehouse comercial. No obstante, reproduce una separación esencial: el fichero Parquet original actúa como activo de datos inmutable, mientras PostgreSQL aloja estructuras depuradas y modelos de consumo. Esta decisión también reduce el volumen

de escritura y evita convertir Bronze en una copia relacional de información que ya dispone de un formato columnar eficiente.

3.3 Arquitectura de Medallón

La arquitectura de medallón organiza el flujo de datos en niveles de refinamiento progresivo, habitualmente denominados Bronze, Silver y Gold. La documentación de Databricks describe Bronze como la capa de datos de origen, Silver como el nivel de validación y limpieza, y Gold como la capa orientada a consumo y agregación [13]. El valor de este patrón no depende de una tecnología concreta, sino de separar responsabilidades.

En este TFM, Bronze se materializa como ficheros Parquet acompañados por metadatos de control. Silver representa el contrato analítico común de los viajes y de las fuentes de referencia. Gold contiene dimensiones, hechos y *marts*. Esta adaptación se diferencia de implementaciones donde las tres capas residen en el mismo motor, pero mantiene el principio de que un dato no se considera listo para negocio hasta haber superado un conjunto explícito de transformaciones y validaciones.

Un aspecto importante es evitar que la arquitectura de medallón se convierta en una mera nomenclatura de carpetas. Para que la separación tenga valor, cada capa debe tener reglas de entrada y salida. Bronze no corrige el contenido del fichero; Silver no incorpora cálculos de negocio que puedan variar por consumidor; Gold no actúa como almacén de datos crudos. Estas fronteras simplifican la depuración y permiten localizar el origen de un error.

3.4 Modelado dimensional

El modelado dimensional organiza métricas cuantitativas en tablas de hechos y atributos descriptivos en dimensiones. Kimball y Ross destacan la importancia de declarar la granularidad de la tabla de hechos antes de diseñar dimensiones y medidas [23]. Esta regla resulta especialmente relevante en movilidad porque el mismo dominio admite múltiples granularidades: un viaje individual, una combinación zona-hora, una combinación origen-destino o una agregación diaria.

El proyecto mantiene una tabla de hechos de viaje cuando la fuente y el volumen lo hacen viable y construye hechos agregados para los principales casos de uso. La tabla `fact_zone_hourly_demand`, por ejemplo, tiene una fila por servicio, zona y hora. Esa granularidad coincide con la variable objetivo del modelado predictivo y evita que cada experimento tenga que volver a agrupar el nivel transaccional.

La capa Gold también incorpora dimensiones conformadas. Una misma `dim_taxi_zone` puede filtrar múltiples hechos y una misma `dim_date` proporciona atributos de calendario consistentes. Esta reutilización es relevante para Superset porque reduce divergencias entre dashboards.

3.5 DataOps, versionado y reproducibilidad

DataOps traslada principios de ingeniería de software a los pipelines de datos: automatización, control de versiones, pruebas, observabilidad y ciclos de cambio controlados. Aunque el término abarca prácticas organizativas amplias, en el contexto de este trabajo se materializa mediante decisiones concretas: Git como historial de código, Docker como

encapsulamiento del entorno, NiFi para controlar el flujo de ingesta, dbt para dependencias y tests de transformación, y tablas de control para registrar ejecuciones.

La reproducibilidad exige algo más que publicar scripts. Una ejecución depende de versiones de librerías, variables de entorno, datos de entrada, parámetros de flujo y configuración. Docker Compose permite declarar los servicios que forman la aplicación y sus relaciones en un fichero versionado [15]. El repositorio debe incluir además un `.env.example`, instrucciones de inicialización, una definición reproducible del flujo NiFi y un modo demo de tamaño razonable.

3.6 Plataformas de flujo de datos y Apache NiFi

Cuando la ingesta crece en número de fuentes y estados, una colección de scripts programados puede requerir código adicional para planificación, reintentos, colas, enrutamiento y trazabilidad. Apache NiFi aborda estas responsabilidades mediante un modelo de programación por flujos en el que los componentes se conectan formando grafos dirigidos y los objetos procesados se representan como FlowFiles [2].

NiFi incorpora *Process Groups* para modularizar el canvas, *Parameter Contexts* para separar configuración de la lógica y estrategias de planificación temporales o CRON [7]. Su capacidad de *Data Provenance* registra el recorrido y las operaciones aplicadas sobre los FlowFiles, permitiendo consultar eventos y representar el linaje [7]. Estas características resultan relevantes en un proyecto académico donde no solo interesa que la ingesta funcione, sino demostrar cómo se controla, se recupera y se audita.

La adopción de NiFi no implica trasladar toda transformación a una herramienta visual. En este trabajo se aplica una separación deliberada: NiFi gobierna movimiento y control operacional; dbt mantiene la lógica SQL de Silver y Gold; Python conserva el modelado predictivo. Esta combinación evita dos extremos: un orquestador Python excesivamente artesanal y un canvas NiFi sobrecargado con lógica analítica que resulta más mantenible en código declarativo.

3.7 Calidad de datos

La calidad de datos puede analizarse mediante dimensiones como completitud, validez, unicidad, consistencia e integridad. En un dataset masivo de viajes, eliminar silenciosamente registros que incumplen una regla dificulta auditar la transformación. Por ello, el proyecto combina tres niveles de control.

El primer nivel valida el fichero antes de su procesamiento: disponibilidad, tamaño, legibilidad y esquema mínimo. El segundo aplica reglas semánticas durante la construcción de Silver: fechas ordenadas, identificadores de zona válidos, duración no negativa o importes coherentes cuando estén disponibles. El tercer nivel utiliza tests de dbt para comprobar propiedades de los modelos relacionales, como claves no nulas, unicidad e integridad referencial.

En la arquitectura basada en NiFi, los controles físicos y de ruta se ejecutan durante la ingesta y sus resultados se registran antes de promover un activo. Las reglas semánticas y relacionales permanecen en dbt. Great Expectations se evaluó como complemento para contratos declarativos [19], pero no se incorpora como dependencia estructural mientras sus comprobaciones puedan cubrirse de forma clara con NiFi, SQL y tests de dbt. Esta decisión reduce solapamientos y mantiene una responsabilidad reconocible para cada herramienta.

3.8 Datos de taxi como fuente para estudiar movilidad

La apertura de los registros de la TLC ha generado una extensa literatura. Hochmair analizó patrones temporales y espaciales, incluyendo el papel de aeropuertos y diferencias entre días laborables y fines de semana [20]. Riascos y Mateos modelaron la red de movilidad a partir de más de mil millones de viajes y construyeron medidas de relevancia basadas en matrices origen-destino [33]. Estos trabajos evidencian que el dataset permite estudiar tanto patrones locales como estructura global de la ciudad.

La irrupción de plataformas de *ride-hailing* amplió el interés hacia la comparación entre taxis tradicionales y servicios de aplicaciones. Faghih et al. analizaron predicción de demanda de Uber en Nueva York mediante modelos espacio-temporales [16]. Toman et al. estudiaron conjuntamente taxis y servicios de *ridesourcing* a nivel de zonas TLC y observaron dependencias espaciales significativas [37]. Este TFM se sitúa en esa línea comparativa, pero incorpora una contribución adicional de ingeniería: la misma plataforma debe ser capaz de integrar varias familias de servicio de forma reproducible.

3.9 Predicción de demanda de taxi

La predicción de demanda es un problema espacio-temporal. El número de viajes de una zona en una hora depende del comportamiento reciente, de periodicidades y de características de la zona. Los métodos propuestos en la literatura abarcan modelos estadísticos autorregresivos, regresiones penalizadas, árboles de decisión, ensembles y redes neuronales profundas.

Safikhani et al. propusieron modelos STAR generalizados con regularización para demanda de Yellow Taxi, mostrando la utilidad de incorporar estructura espacio-temporal [34]. Faghih et al. estudiaron el corto plazo en demanda de Uber y compararon modelos temporales y espacio-temporales [16]. Trabajos posteriores han utilizado Random Forest, XGBoost y otras técnicas de machine learning para capturar relaciones no lineales [12]. También existe una línea de investigación basada en redes LSTM, convoluciones y mecanismos de atención [25, 11].

El presente proyecto no persigue competir con arquitecturas profundas de estado del arte. La decisión es deliberada: el objetivo académico incluye comparar modelos representativos, interpretables y ejecutables en un equipo local con 32 GB de RAM. Se prioriza un diseño experimental reproducible sobre una búsqueda exhaustiva de complejidad. Por esta razón se eligen tres familias con sesgos inductivos diferentes.

3.10 Regresión de Poisson como referencia estructurada

La demanda es una variable de conteo no negativa. La regresión de Poisson modela la esperanza condicional mediante un enlace logarítmico:

$$\log(\mathbb{E}[Y | X]) = \beta_0 + X\beta. \quad (3.1)$$

Su principal ventaja es ofrecer una referencia lineal coherente con la naturaleza de la variable objetivo. No se asume que sea el mejor modelo, especialmente si existe sobredispersión, interacciones complejas o cambios de régimen. Su función es establecer un punto de comparación interpretable y revelar cuánto rendimiento adicional aportan los modelos no lineales. La implementación se basa en `PoissonRegressor` de `scikit-learn` [35, 31].

3.11 Random Forest

Random Forest combina múltiples árboles entrenados sobre muestras y subconjuntos de variables para reducir la varianza de un único árbol [10]. En datos tabulares puede capturar interacciones no lineales sin exigir una especificación funcional previa. Además, proporciona medidas de importancia de variables y una base conocida para comparar con métodos de boosting.

Su principal limitación en este proyecto es computacional. El histórico por zona y hora puede generar millones de filas, y un bosque con muchos árboles consume memoria de forma significativa. Por ello, el diseño experimental debe documentar el muestreo, número de árboles, profundidad y restricciones de nodos utilizadas.

3.12 Gradient Boosting y LightGBM

Los métodos de gradient boosting construyen árboles secuencialmente para corregir errores de los modelos anteriores [17]. LightGBM introduce optimizaciones orientadas a grandes conjuntos tabulares y utiliza algoritmos de histogramas y crecimiento de árbol eficientes [22, 24]. Estas propiedades lo convierten en un candidato natural para el volumen de este TFM.

La comparación entre Random Forest y LightGBM también tiene interés metodológico: uno representa *bagging* y otro *boosting*. Si LightGBM obtiene mejor resultado, será necesario comprobar que la mejora compensa su mayor complejidad de ajuste y que se mantiene de manera consistente entre servicios y horizontes.

3.13 Validación temporal y fuga de información

En predicción de series temporales, una partición aleatoria convencional puede situar observaciones futuras en entrenamiento y observaciones pasadas en test. Esto produce una evaluación optimista, especialmente cuando se utilizan variables *lag* o medias móviles. El proyecto aplica divisiones cronológicas: el entrenamiento contiene únicamente observaciones anteriores a validación y test.

Las variables derivadas deben seguir la misma regla. Por ejemplo, una media móvil de 24 horas para predecir $t + 1$ puede utilizar $t, t - 1, \dots, t - 23$, pero nunca información posterior. La implementación de features se diseña para que la fecha de corte sea explícita y verificable.

3.14 Métricas de evaluación

Se utilizan métricas complementarias porque ninguna resume por sí sola el comportamiento del modelo. El error absoluto medio se define como:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (3.2)$$

El RMSE penaliza con mayor intensidad errores grandes:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3.3)$$

Finalmente, WAPE proporciona un error porcentual agregado robusto ante la presencia de observaciones individuales iguales a cero:

$$\text{WAPE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|}. \quad (3.4)$$

El coeficiente R^2 se incluye como métrica secundaria, pero no se utiliza de forma aislada para seleccionar el modelo.

3.15 Posicionamiento del presente trabajo

La literatura demuestra que los registros de taxi son adecuados para estudiar movilidad y que la demanda presenta estructura espacial y temporal. Sin embargo, gran parte de los trabajos se concentra en el algoritmo predictivo o en un análisis específico. Este TFM adopta una perspectiva de producto analítico: el problema comienza en la fuente de datos y finaliza en una capa de consumo reproducible.

La contribución no pretende ser un nuevo algoritmo de forecasting. Se sitúa en la integración rigurosa de prácticas de ingeniería de datos con análisis y machine learning: múltiples fuentes TLC, arquitectura de medallón, modelado dimensional, calidad declarativa, ejecución contenerizada, comparación temporal de modelos y publicación en dashboards. Esta combinación permite evaluar no solo si una predicción es precisa, sino si puede reproducirse y explicarse dentro de un sistema completo.

CAPÍTULO 4

Marco tecnológico

4.1 Criterios de selección

La selección tecnológica se guía por reproducibilidad, mantenibilidad, trazabilidad, capacidad de ejecución en un único equipo y disponibilidad de herramientas abiertas o de libre utilización. El objetivo no es acumular tecnologías, sino asignar una responsabilidad concreta a cada componente. La arquitectura final separa el movimiento y control operacional del dato, la transformación analítica, el aprendizaje automático y la visualización.

La solución se ejecuta sobre Windows con Docker Desktop, preferiblemente mediante integración WSL2. Los procesos de aplicación se encapsulan en contenedores Linux, reduciendo la dependencia del sistema anfitrión y permitiendo que el mismo repositorio pueda trasladarse posteriormente a un servidor Linux con cambios mínimos.

4.2 Docker y Docker Compose

Docker proporciona el mecanismo de aislamiento utilizado para empaquetar dependencias y procesos. Cada servicio se define mediante una imagen y un conjunto explícito de variables, volúmenes, redes y comprobaciones de salud. Docker Compose permite describir y ejecutar aplicaciones formadas por varios contenedores desde una especificación declarativa [15].

En este trabajo, Compose reproduce la relación entre Apache NiFi, PostgreSQL, dbt, el entorno Python de machine learning, Redis y Superset. La plataforma debe poder arrancarse desde la raíz del repositorio mediante:

```
1 docker compose up -d
```

Listing 4.1: Arranque general del entorno contenerizado.

Los directorios Bronze, los repositorios internos de NiFi que requieran persistencia, los modelos entrenados y la base de datos sobreviven a la recreación de los contenedores mediante volúmenes o montajes explícitos. Las credenciales de desarrollo se externalizan en variables de entorno o mecanismos equivalentes y nunca se incorporan al repositorio público.

4.3 Apache NiFi

Apache NiFi constituye la capa principal de ingesta y automatización del flujo de datos. Se trata de un sistema basado en programación por flujos que permite construir grafos dirigidos de enrutamiento, transformación e intermediación mediante una interfaz web [2]. En la arquitectura del TFM se utiliza para descubrir activos, realizar peticiones HTTP, controlar el estado de ficheros, enrutar errores, escribir Bronze, coordinar accesos a PostgreSQL y desencadenar las fases posteriores.

La elección de NiFi responde especialmente a cuatro capacidades. En primer lugar, cada procesador puede configurarse con planificación temporal o basada en expresiones CRON, evitando mantener un scheduler separado para la ingesta [7]. En segundo lugar, los *Process Groups* permiten encapsular subflujos y expresar la arquitectura de forma modular. En tercer lugar, los *Parameter Contexts* desacoplan configuración de la lógica del flujo y permiten distinguir parámetros sensibles y no sensibles. Finalmente, el subsistema de *Data Provenance* registra eventos asociados al recorrido de los FlowFiles, facilitando diagnóstico, linaje y auditoría [7].

La implementación organiza el canvas mediante grupos de procesos dedicados a control, descubrimiento, descarga, validación, fuentes externas, ejecución de dbt, disparo del proceso de ML y tratamiento de errores. Esta estructura se desarrolla con el propósito de que el flujo pueda entenderse tanto a nivel operativo como en la defensa académica del proyecto.

4.4 NiFi y PostgreSQL

NiFi se conecta a PostgreSQL mediante servicios JDBC administrados como *Controller Services*. La documentación oficial proporciona servicios de *connection pooling* y procesadores capaces de ejecutar SQL o escribir registros en una base relacional [1, 4]. En el TFM, PostgreSQL es la fuente de verdad para el estado persistente del pipeline: ejecuciones, activos descubiertos, estados de ingesta y resultados de calidad.

Esta decisión evita depender exclusivamente del estado interno de NiFi. La procedencia de NiFi aporta trazabilidad del recorrido de cada FlowFile, mientras que las tablas del esquema control ofrecen una vista relacional, estable y consultable de las ejecuciones. Ambos mecanismos son complementarios.

4.5 Apache Parquet

Parquet es un formato columnar diseñado para almacenamiento y recuperación eficientes [8]. Su utilización en Bronze está condicionada por la propia fuente, ya que la TLC distribuye los registros mensuales en este formato [29]. Mantener el fichero original evita una conversión temprana innecesaria y permite explotar lectura selectiva de columnas y compresión.

La estructura de carpetas particiona lógicamente por dominio, servicio, año y mes. NiFi escribe cada activo únicamente cuando la transferencia y las validaciones mínimas han finalizado correctamente. La presencia física del fichero se acompaña siempre de metadatos en PostgreSQL.

4.6 PostgreSQL

PostgreSQL actúa como almacén relacional central para las capas Silver y Gold, además de contener los esquemas de control, auditoría y cuarentena. Se elige por su madurez, soporte SQL, ecosistema, capacidad de indexación y facilidad de integración con Docker [32].

La solución utiliza varios esquemas lógicos, previsiblemente *control*, *silver*, *gold* y *quarantine*. PostgreSQL no se utiliza como almacenamiento primario de Bronze, porque duplicar todos los ficheros originales incrementaría de forma sustancial el espacio utilizado sin aportar valor equivalente.

4.7 dbt

data build tool (dbt) se utiliza para transformar datos accesibles al motor relacional. dbt permite definir modelos SQL con dependencias explícitas, ejecutar pruebas y generar documentación del grafo de transformación [14]. De este modo, la lógica Silver y Gold permanece declarativa, versionada y separada del flujo operacional de NiFi.

La organización distingue modelos de *staging*, Silver, Gold y *marts*. NiFi puede desencadenar una ejecución de dbt cuando el conjunto de entrada requerido está disponible, pero no absorbe la lógica analítica que corresponde a los modelos SQL. Esta separación evita construir transformaciones complejas mediante un canvas visual cuando pueden expresarse con mayor claridad en SQL versionado.

4.8 Python

Python se mantiene como lenguaje principal de la capa de machine learning y para utilidades que requieran lógica de propósito general. Se utiliza para preparación de features no resueltas en SQL, entrenamiento, validación temporal, cálculo de métricas, persistencia de artefactos e inferencia.

Las primeras pruebas de ingesta y orquestación se realizaron con scripts Python. Esta implementación se conserva en el repositorio bajo una carpeta independiente, *Ingesta Python/*, como evidencia de la evolución arquitectónica y como referencia funcional. Tras validar la viabilidad del pipeline, se optó por Apache NiFi para la ingesta principal, al ofrecer una solución más reproducible y mantenible para programar, enrutar, reintentar y supervisar los flujos. Python se mantiene en la arquitectura para la preparación de variables, el entrenamiento y la inferencia de los modelos.

4.9 scikit-learn

scikit-learn proporciona una interfaz consistente para preprocesamiento, evaluación y modelos clásicos de aprendizaje automático [31]. En este trabajo se utiliza para la regresión de Poisson, Random Forest, métricas y utilidades de pipeline.

La selección de *PoissonRegressor* responde a la naturaleza de conteo del objetivo [35]; *RandomForestRegressor* representa un ensemble de bagging capaz de modelar relaciones no lineales [36].

4.10 LightGBM

LightGBM es una implementación de gradient boosting basada en árboles optimizada para datos tabulares y grandes volúmenes [22, 24]. Su API compatible con scikit-learn facilita incorporarlo al mismo marco de entrenamiento y evaluación.

La elección de LightGBM no presupone que sea el modelo ganador. Los resultados experimentales deberán justificar si su complejidad adicional se traduce en una mejora suficiente respecto a Random Forest, Poisson y el baseline estacional.

4.11 Apache Superset

Apache Superset es una plataforma de exploración y visualización de datos que puede conectarse a múltiples bases de datos y construir dashboards interactivos [9]. En este proyecto consume exclusivamente modelos Gold o vistas específicamente preparadas para BI. Se evita que los gráficos contengan lógica de transformación sustantiva que no exista también en el almacén de datos.

Esta decisión mejora la trazabilidad: cada KPI debe tener una definición reproducible en SQL/dbt y Superset se utiliza para presentar, filtrar y combinar los resultados.

4.12 Redis

Redis se incorpora como servicio auxiliar de Superset cuando la configuración seleccionada requiere caché o componentes asíncronos. No forma parte del modelo analítico ni contiene la fuente de verdad del proyecto.

4.13 Git, GitHub y versionado de flujos

Git registra la evolución del código y permite asociar decisiones y resultados a revisiones concretas [18]. El repositorio público se utiliza además como mecanismo de entrega reproducible. El historial debe excluir secretos, ficheros de datos masivos y volúmenes de ejecución.

El versionado del flujo NiFi se diseña utilizando las capacidades de *Flow Registry Client* basadas en Git introducidas en NiFi 2. La propia comunidad de Apache NiFi anunció en 2026 la deprecación de NiFi Registry y recomienda migrar hacia clientes integrados con GitHub, GitLab, Bitbucket o Azure DevOps [5]. En este proyecto se prioriza la integración con GitHub para evitar desplegar un servicio Registry adicional y mantener el flujo junto al resto de artefactos del TFM.

Los valores sensibles utilizados por los clientes de registro no se almacenan en Git. La configuración versionada debe conservar la estructura del flujo y los parámetros no secretos, mientras que tokens y credenciales permanecen externalizados.

4.14 Relación entre componentes

La figura 4.1 resume las responsabilidades. NiFi recibe y enruta los datos de las fuentes, preserva Bronze y registra el estado operacional. dbt transforma las capas Silver y

Gold sobre PostgreSQL. Python consume features Gold para entrenar y ejecutar modelos. Superset consulta marts y predicciones persistidas.

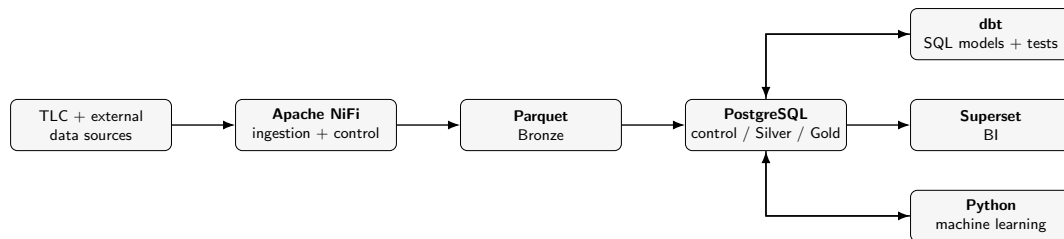


Figura 4.1: Relación funcional entre las principales tecnologías del proyecto con Apache NiFi como capa de ingesta y automatización.

4.15 Tecnologías deliberadamente no incorporadas

No se introduce Airflow, Prefect o Dagster en la arquitectura principal. NiFi cubre la planificación y coordinación del flujo de ingesta, mientras que dbt mantiene el DAG de transformaciones SQL y Python se ocupa del ciclo de machine learning. Añadir un segundo orquestador incrementaría la superficie operativa sin una necesidad demostrada para el alcance local del proyecto.

Great Expectations fue considerado como herramienta adicional de contratos de calidad. En la versión NiFi se prioriza una distribución más simple de responsabilidades: validaciones físicas y de ruta en NiFi, reglas semánticas y tests relacionales en dbt, y validaciones específicas de ML en Python. Si durante la implementación se detecta una necesidad no cubierta, GX puede incorporarse de forma selectiva, pero no se considera requisito estructural.

CAPÍTULO 5

Fuentes de datos y análisis preliminar

5.1 Fuente principal: TLC Trip Record Data

La fuente principal del trabajo es el portal de *Trip Record Data* de la New York City Taxi and Limousine Commission. La página oficial ofrece descargas mensuales en formato Parquet y diccionarios de datos diferenciados por servicio [29]. La TLC advierte además de que los datos se publican a partir de registros remitidos por operadores y bases, por lo que la publicación no debe interpretarse como garantía absoluta de exactitud de cada observación [28].

Esta característica refuerza la necesidad de una capa propia de calidad. El pipeline no modifica el fichero Bronze, pero registra incidencias de esquema y aplica reglas de validez antes de promover datos a Silver.

5.2 Periodo de estudio

La ventana general comprende desde el 1 de enero de 2018 hasta el 31 de diciembre de 2025. La elección permite dividir conceptualmente el análisis en tres fases:

- **Pre-pandemia:** 2018–2019, utilizado como referencia de comportamiento previo.
- **Disrupción y transición:** 2020–2021, periodo en el que la movilidad experimenta cambios extraordinarios.
- **Recuperación y nuevo equilibrio:** 2022–2025, donde se observa la evolución posterior y la consolidación de nuevas pautas.

La división es analítica, no causal. El trabajo no presupone que todo cambio entre periodos se deba a la pandemia.

5.3 Disponibilidad por servicio

La cuadro 5.1 resume el alcance previsto. La cobertura exacta se verificará automáticamente contra el portal durante la ejecución.

Tabla 5.1: Servicios TLC incluidos y cobertura analítica prevista.

Servicio	2018	2019–2025	Tarifas	Uso principal
Yellow	Sí	Sí	Amplias	Movilidad, economía y predicción
Green	Sí	Sí	Amplias	Movilidad, economía y predicción
FHV	Sí	Sí	Limitadas/variables	Movilidad y predicción
HV FHV	No separado	Sí	Esquema propio	Movilidad, comparación y predicción

Desde 2019 los HV FHV se reportan como un dataset separado y más detallado [29]. En consecuencia, cualquier gráfico comparativo que incluya esta modalidad debe comenzar en 2019 o marcar explícitamente la falta de cobertura anterior.

5.4 Unidad de ingesta

La unidad de ingesta es el fichero mensual por servicio. Cada activo se identifica mediante una clave lógica compuesta por servicio, año, mes y URL de origen. Cuando sea viable se calcula además un hash criptográfico para detectar cambios en un fichero publicado con el mismo nombre.

Los metadatos mínimos son:

- servicio;
- año y mes;
- URL;
- ruta local;
- tamaño en bytes;
- hash;
- fecha de descubrimiento y descarga;
- estado de validación;
- estado de promoción a Silver;
- número de registros leído y aceptado.

5.5 Cambios de esquema

Un histórico de ocho años no debe asumirse esquemáticamente estable. Los diccionarios TLC evolucionan y los campos disponibles dependen del servicio y periodo. Por ejemplo, la página oficial indica que a partir de 2025 se añadió un campo de cargo de congestión CBD en determinados datasets [29]. Si el pipeline utilizara una lista rígida de columnas para todo el histórico, una incorporación de este tipo podría provocar errores o, peor aún, pérdidas silenciosas de información.

La estrategia adoptada separa tres conceptos:

1. **Esquema físico observado:** columnas y tipos contenidos en cada Parquet.
2. **Contrato Silver común:** conjunto de atributos homologados necesarios para análisis transversal.
3. **Atributos específicos:** campos propios de un servicio o periodo que se conservan cuando aportan valor.

El contrato Silver permite que un consumidor conozca qué significa cada campo sin tener que estudiar cada versión del fichero original.

5.6 Taxonomía común del viaje

Se propone un núcleo de campos comunes como el mostrado en la cuadro 5.2. La existencia real de cada atributo se verificará por familia y año.

Tabla 5.2: Contrato conceptual común para un viaje Silver.

Campo	Tipo	Significado
service_type	texto	Modalidad TLC de procedencia.
pickup_datetime	timestamp	Instante de inicio del viaje o recogida.
dropoff_datetime	timestamp	Instante de finalización cuando esté disponible.
pickup_location_id	entero	Zona TLC de origen.
dropoff_location_id	entero	Zona TLC de destino.
trip_distance	numérico	Distancia declarada cuando exista.
trip_duration_minutes	numérico	Duración derivada entre marcas temporales.
base_fare	numérico	Tarifa base cuando la fuente la proporcione.
total_amount	numérico	Importe total observado cuando sea comparable.
source_file	texto	Fichero Bronze del que procede el registro.
source_year/month	entero	Partición temporal de procedencia.

5.7 Taxi Zone Lookup y geometrías

La guía de usuario de la TLC proporciona una tabla de zonas y un fichero geográfico para representar las áreas utilizadas en los registros [30]. Se incorpora una dimensión `dim_taxi_zone` con identificador, nombre de zona, borough y *service zone*. Las geometrías se conservan en un activo geográfico apropiado para construir mapas.

El uso de la taxonomía oficial evita inventar cuadrículas arbitrarias y facilita que los resultados puedan compararse con otros análisis basados en TLC zones.

5.8 Calendario y festivos

La dimensión de calendario se genera de forma determinista y no requiere un fichero masivo externo. Para cada fecha se incluyen, como mínimo, año, trimestre, mes, semana, día de la semana, indicador de fin de semana, estación y festivo. Los festivos se parametrizan con una fuente o librería versionada y deben quedar materializados en Gold para que un cambio de dependencia no modifique silenciosamente un experimento histórico.

5.9 Meteorología

Se prevé utilizar datos históricos de NOAA/NCEI para una estación representativa de Nueva York o una combinación documentada de estaciones [27]. La decisión final debe equilibrar disponibilidad temporal y simplicidad de unión. Para el objetivo predictivo, la granularidad preferente es horaria.

Las variables candidatas incluyen:

- temperatura;
- precipitación;
- nieve;
- velocidad del viento;
- visibilidad;
- indicadores derivados de condiciones adversas.

Si una variable solo está disponible con granularidad diaria, se documentará para evitar dar a entender una precisión temporal inexistente.

5.10 Reglas iniciales de calidad

Antes de construir indicadores se aplican reglas generales y específicas. Entre las reglas generales se encuentran:

- marcas temporales dentro de rangos plausibles;
- fecha de bajada posterior o igual a la de recogida cuando ambas existan;
- duración no negativa;
- distancias no negativas;
- importes no negativos salvo campos donde un ajuste contable pueda justificarlo;
- zonas de origen y destino pertenecientes al catálogo cuando el campo exista;
- ausencia de duplicados según la clave técnica definida;
- coherencia entre el mes del fichero y la fecha principal, permitiendo una tolerancia documentada en viajes que cruzan límites temporales.

Una regla que falla no implica siempre que el registro se elimine. Se distinguen errores críticos, advertencias y anomalías estadísticas. La causa de exclusión debe quedar registrada.

5.11 Outliers y valores extremos

Los datos de movilidad contienen valores extremos legítimos y errores. Un viaje largo hacia un aeropuerto o fuera de las zonas centrales no debe descartarse únicamente por ser infrecuente. Por esta razón se evita utilizar exclusivamente umbrales estadísticos globales para eliminar observaciones.

La estrategia combina restricciones físicas o semánticas con percentiles utilizados como señal de inspección. Los umbrales definitivos se documentarán tras el análisis exploratorio y se comparará el efecto de filtrado sobre la distribución. El objetivo es evitar que la limpieza elimine precisamente aquellos eventos que explican parte de la heterogeneidad urbana.

5.12 Análisis exploratorio previsto

El análisis exploratorio se estructura en cuatro bloques. El primero cuantifica volumen y cobertura de datos. El segundo estudia distribución temporal. El tercero examina geografía y flujos. El cuarto analiza campos económicos allí donde sean comparables.

5.12.1. Volumen y cobertura

Se reportarán número de ficheros, tamaño, registros y porcentaje de filas válidas por servicio y año. En la versión final, la plataforma habrá procesado pendiente viajes y un volumen Bronze de pendiente.

Figura pendiente de generar: Serie mensual del número de viajes por servicio entre 2018 y 2025, generada a partir de `mart_mobility_overview`.

Figura 5.1: Evolución mensual del volumen de viajes por tipo de servicio.

5.12.2. Patrones temporales

Se analizarán perfiles horarios, diferencias laborable/fin de semana, estacionalidad mensual y periodos de ruptura. Es importante representar tanto valores absolutos como índices relativos, dado que la escala entre servicios puede diferir varios órdenes de magnitud.

5.12.3. Patrones geográficos

Los mapas mostrarán densidad de recogidas por zona y cambios relativos entre periodos. También se construirán matrices o rankings de pares origen-destino, evitando visualizaciones que resulten ilegibles por el número de combinaciones.

5.12.4. Indicadores económicos

Para Yellow y Green se estudiarán distribuciones de importe, propina, distancia, duración e ingreso observado por hora ocupada. Estas métricas se utilizarán para el dashboard de oportunidades, pero siempre acompañadas de tamaño muestral para evitar rankings dominados por zonas con pocas observaciones.

5.13 Trazabilidad de la fuente al indicador

Cada tabla Gold conserva las claves necesarias para conocer la procedencia de las agregaciones. En el nivel transaccional se conserva `source_file`; en el nivel agregado se mantiene el periodo y servicio. La tabla de control permite reconstruir qué ficheros participaron en una ejecución determinada. Esta trazabilidad es necesaria tanto para depuración como para defensa académica de los resultados.

CAPÍTULO 6

Diseño de la arquitectura

6.1 Requisitos de arquitectura

La arquitectura se diseña a partir de requisitos funcionales y no funcionales. Entre los funcionales se encuentran descubrir y descargar datos, conservar el dato de origen, transformar y validar los registros, generar modelos analíticos, entrenar modelos predictivos y servir dashboards. Entre los no funcionales destacan reproducibilidad, idempotencia, trazabilidad, mantenibilidad, recuperación ante fallos y capacidad de ejecución con recursos locales.

El equipo objetivo dispone de Windows, 32 GB de RAM, procesador AMD Ryzen, aproximadamente 1 TB de almacenamiento total y Docker Desktop. Esta configuración obliga a controlar el uso de memoria y evita asumir un clúster distribuido. La solución debe trabajar tanto con un modo demo reducido como con un modo completo que procese el histórico configurado.

6.2 Evolución de la arquitectura

La primera iteración del proyecto utilizaba scripts Python para descarga, validación inicial y coordinación de tareas, junto con planificación mediante cron. Ese enfoque permitió materializar rápidamente el pipeline y validar la viabilidad funcional. Sin embargo, a medida que aumentaron las fuentes, estados y requisitos de observabilidad, la lógica de control comenzó a adquirir un peso comparable al procesamiento de datos.

La arquitectura final migra la capa de ingesta y automatización hacia Apache NiFi. El objetivo no es sustituir Python de forma indiscriminada, sino utilizar una herramienta especializada allí donde aporta mayor valor: programación del flujo, enrutamiento, reintentos, configuración, visualización operacional y procedencia. La implementación Python original se conserva como referencia en Ingesta Python/; dbt, PostgreSQL, Superset y el código de machine learning permanecen en la arquitectura principal.

6.3 Comparación de alternativas de ingesta

La decisión de migrar no se basa únicamente en preferencia tecnológica. La cuadro 6.1 resume las diferencias relevantes para el alcance del TFM.

Tabla 6.1: Comparación entre la implementación inicial y la arquitectura NiFi.

Criterio	Python + cron	Apache NiFi
Control del flujo	Código explícito y flexible, pero requiere implementar el coordinador	Grafo visual con relaciones y colas explícitas
Planificación	Cron externo	Timer/CRON integrado por procesador
Reintentos	Lógica propia	Rutas y políticas configurables dentro del flujo
Trazabilidad	Logs y tablas de control	Logs, tablas de control y Data Provenance
Configuración	Variables/YAML propios	Parameter Contexts y variables de entorno
Versionado	Código Git convencional	Definición de flujo versionable mediante clientes Git
Coste operativo	Menor huella base	Mayor consumo de memoria y repositorios internos

La alternativa Python continúa siendo técnicamente válida y se conserva como evidencia de la primera fase. NiFi se selecciona porque reduce la cantidad de infraestructura de control implementada específicamente para el TFM y hace visible el estado del flujo sin trasladar a NiFi la transformación analítica ni el machine learning.

6.4 Visión general

La figura 6.1 representa el flujo lógico. Apache NiFi obtiene los activos TLC y las fuentes externas, registra su estado y preserva los Parquet originales en Bronze. La carga relacional y dbt construyen Silver y Gold. El módulo de machine learning consume features Gold y persiste predicciones y métricas. Superset consulta únicamente modelos preparados para consumo.

Figura pendiente de generar: architecture_overview.pdf

Figura 6.1: Arquitectura lógica end-to-end con Apache NiFi como capa de ingesta y automatización.

6.5 Capa Bronze

Bronze representa la copia más fiel posible de la fuente publicada. NiFi no reescribe los archivos descargados para homogeneizarlos. Si se requiere una representación transformada para acelerar una fase posterior, dicha representación pertenece a Silver o a un área temporal claramente separada.

La estructura física propuesta es:

```

1 data/
2   bronze/
3     tlc/

```

```

4     yellow/year=YYYY/month=MM/
5     green/year=YYYY/month=MM/
6     fhv/year=YYYY/month=MM/
7     fhvhv/year=YYYY/month=MM/
8     weather/
9     geography/
10    calendar/
11    quarantine/
12    samples/

```

Listing 6.1: Estructura lógica del almacenamiento Bronze.

La tabla `control.ingestion_files` registra el ciclo de vida de cada activo. La existencia de una ruta en disco no es suficiente para considerar un fichero correctamente procesado.

6.6 Capa Silver

Silver contiene datos tipados, validados y semánticamente normalizados. El principal reto es integrar esquemas diferentes sin eliminar información valiosa. Se utiliza una estrategia de modelos por servicio en staging y una proyección común en Silver.

Cada familia puede exponer un modelo de staging propio, por ejemplo `stg_yellow_trips`, junto con modelos equivalentes para Green, FHV y HVFHV. Los campos comunes se proyectan a una estructura homologada mediante reglas explícitas. Los campos económicos ausentes se representan como nulos y no como cero, porque cero tendría un significado cuantitativo distinto de *no disponible*.

6.7 Capa Gold

Gold contiene modelos orientados a consumo. Se combinan un modelo dimensional y *data marts* agregados. El modelo dimensional favorece exploración flexible; los marts optimizan consultas recurrentes y fijan definiciones de KPI.

La arquitectura inicial incluye:

- `dim_date;`
- `dim_time;`
- `dim_taxi_zone;`
- `dim_service_type;`
- `dim_payment_type;`
- `dim_weather;`
- `fact_trip;`
- `fact_zone_hourly_demand;`
- `fact_zone_daily_performance;`
- `fact_model_predictions;`
- `fact_model_metrics;`
- marts de movilidad, comparación, oportunidad y monitorización predictiva.

6.8 Modelo dimensional

La granularidad de `fact_trip` es un viaje validado. La de `fact_zone_hourly_demand` es una combinación de servicio, zona de recogida y hora. Esta segunda tabla es fundamental porque alinea el almacén analítico con el dataset de machine learning.

Figura pendiente de generar: star_demand.pdf

Figura 6.2: Esquema estrella simplificado para la demanda horaria por zona.

6.9 NiFi como capa de ingesta y automatización

NiFi organiza el flujo mediante *Process Groups*. La separación propuesta distingue control del pipeline, descubrimiento TLC, descarga, validación Bronze, ingestión de fuentes externas, activación de dbt, activación de ML y tratamiento de errores. Los procesadores se conectan mediante relaciones explícitas y colas, de manera que cada transición sea visible y monitorizable [7].

Figura pendiente de generar: nifi_process_groups.pdf

Figura 6.3: Organización conceptual de los grupos de procesos de Apache NiFi.

La arquitectura utiliza el motor tradicional de ejecución para los grupos que requieren planificación CRON o persistencia de colas. NiFi permite definir planificación *Timer Driven* y *CRON Driven*, así como controlar la concurrencia de los procesadores [7]. Esta capacidad sustituye el contenedor de cron de la implementación anterior.

6.10 Parameter Contexts

Los valores que cambian entre entornos no se incrustan en los procesadores. Se agrupan mediante *Parameter Contexts*, por ejemplo para TLC, PostgreSQL, rutas, configuración global del pipeline y fuentes externas. La sintaxis de parámetros de NiFi permite referenciarlos desde las propiedades de los componentes y distinguir valores sensibles [7].

Entre los parámetros previstos se encuentran:

- rango temporal de ingesta;
- modo demo o full;
- rutas Bronze y cuarentena;

- host, puerto y base de datos PostgreSQL;
- límites de reintentos y timeouts;
- configuración de fuentes externas.

Las credenciales no se versionan con valores reales.

6.11 Esquema de control

El esquema control evita almacenar el estado únicamente dentro del canvas o de ficheros locales. Se proponen cuatro entidades principales:

`pipeline_runs` una fila por ejecución global, con identificador, parámetros, commit, inicio, fin y estado.

`pipeline_tasks` estado y duración de cada etapa relevante.

`ingestion_files` inventario de activos descubiertos, descargados, validados y procesados.

`data_quality_results` resultado resumido de validaciones y referencia al detalle.

NiFi consulta y actualiza estas entidades mediante conexiones JDBC administradas. La base de datos proporciona un estado persistente independiente de la vida del contenedor.

6.12 Idempotencia

Un proceso es idempotente si repetirlo con la misma entrada no modifica el resultado de forma no deseada. En la arquitectura NiFi, la idempotencia se implementa combinando estado de PostgreSQL, atributos del FlowFile y comprobaciones físicas. Antes de descargar o publicar un activo se consulta su clave lógica y estado; un fichero validado no se transfiere de nuevo salvo que la política detecte una nueva versión o una inconsistencia.

No se confía únicamente en *“si el fichero existe, saltarlo”*. Un fichero puede existir parcialmente, haber cambiado en origen o pertenecer a una ejecución fallida. Por ello, la decisión combina presencia, tamaño, checksum cuando resulte viable, estado y metadatos de procedencia.

6.13 Data Provenance y observabilidad

NiFi registra eventos de procedencia a medida que los FlowFiles atraviesan el sistema. Su interfaz permite consultar qué ocurrió con un objeto, examinar eventos y visualizar su linaje a lo largo del flujo [7]. Esta capacidad complementa las tablas de control y resulta especialmente útil para la defensa del TFM, porque permite demostrar el recorrido de un fichero desde la fuente hasta Bronze y las etapas posteriores.

La observabilidad se apoya en cuatro niveles:

1. estado visual y colas de NiFi;

2. *bulletins* y logs del servicio;
3. *Data Provenance* para seguimiento del FlowFile;
4. tablas control de PostgreSQL para auditoría histórica.

Las métricas operativas incluyen número de ficheros descubiertos, descargados, omitidos y fallidos, bytes transferidos, tiempos por fase y resultados de calidad.

6.14 Reintentos y tratamiento de errores

Los errores transitorios, como timeouts HTTP o indisponibilidad temporal de PostgreSQL, siguen rutas de reintento con límites configurados. Los errores deterministas de esquema o validación no se reintentan indefinidamente. Tras alcanzar el máximo, el activo se marca como FAILED o QUARANTINED y conserva información suficiente para diagnosticar la causa.

La topología mantiene un grupo de procesos específico para errores. Esta separación evita mezclar el camino nominal con reglas de recuperación y permite centralizar métricas y política de reintentos.

6.15 Versionado de los flujos

Los flujos deben poder reconstruirse a partir del repositorio. La estrategia prioriza los clientes de registro basados en Git disponibles en NiFi 2. La página oficial del proyecto NiFi Registry indica que el Registry independiente fue deprecado en 2026 y recomienda los clientes Git para GitHub, GitLab, Bitbucket y Azure DevOps [5].

Para este TFM se adopta GitHub como destino coherente con el repositorio público. Los artefactos de flujo y la configuración no sensible se versionan; tokens, claves o parámetros sensibles quedan fuera del historial.

6.16 Integración con dbt y machine learning

NiFi coordina, pero no absorbe la lógica propia de dbt o del modelado predictivo. Una vez que los activos de entrada requeridos están disponibles y validados, el flujo desencadena la construcción Silver/Gold. dbt continúa siendo responsable del grafo de dependencias SQL y de sus tests.

De forma análoga, el entrenamiento y la inferencia permanecen en Python. NiFi actúa como punto de activación y control, mientras el código ML conserva su estructura de módulos, configuración, métricas y artefactos. Esta separación permite probar cada componente de manera independiente.

6.17 Seguridad y secretos

El repositorio será público. Las credenciales se suministran mediante `.env`, `Parameter Contexts` sensibles o mecanismos equivalentes, y se excluyen del historial Git. El flujo versionado no debe contener tokens de GitHub, contraseñas de PostgreSQL ni credenciales administrativas de NiFi.

Se revisarán también la implementación Python histórica, notebooks y configuraciones en busca de rutas locales, información empresarial o secretos antes de la publicación.

6.18 Modo demo y modo full

El modo demo utiliza una ventana temporal pequeña suficiente para demostrar todas las fases. Debe completar la descarga, validación, transformación, tests, inferencia y disponibilidad de datos para Superset sin requerir cientos de gigabytes.

El modo full procesa el rango configurado desde 2018 hasta 2025 y todos los servicios seleccionados. Ambos modos reutilizan los mismos Process Groups; cambian los Parameter Contexts o parámetros de ejecución. Esta propiedad evita disponer de una demo artificialmente separada de la arquitectura real.

6.19 Reproducibilidad

Se considera que la arquitectura es reproducible cuando una tercera persona puede clonar el repositorio, configurar variables no sensibles, ejecutar Docker Compose, restaurar o importar la definición del flujo y obtener un entorno funcional. Para los resultados experimentales se añade una condición adicional: identificar versiones de datos, código, parámetros y flujos utilizados.

La reproducibilidad se comprobará mediante una prueba end-to-end en modo demo y quedará documentada en el apartado correspondiente del repositorio.

CAPÍTULO 7

Implementación del pipeline de datos

7.1 Organización del repositorio

Las primeras pruebas de la ingesta se hicieron en Python y permitieron validar el acceso a las fuentes y el procesamiento inicial. Finalmente, se optó por una implementación basada en Apache NiFi, más reproducible y mantenible para coordinar la descarga, las validaciones, los reintentos y el seguimiento del estado. La migración mantiene separadas la arquitectura actual y la implementación Python anterior. La estructura objetivo se resume en el listado 7.1.

```
1 tfm-nyc-mobility-platform/  
2   docker-compose.yml  
3   .env.example  
4   Makefile  
5   Ingesta Python/  
6     README.md  
7     src/  
8     tests/  
9   nifi/  
10    flows/  
11    parameters/  
12    sql/  
13    docs/  
14   dbt/  
15    models/  
16      staging/  
17      silver/  
18      gold/  
19      marts/  
20    tests/  
21    macros/  
22   ml/  
23    src/  
24    configs/  
25    artifacts/  
26   superset/  
27    exports/  
28   sql/  
29    init/  
30    control/  
31   docs/  
32   data/  
33    samples/
```

Listing 7.1: Estructura objetivo del repositorio tras la migración a NiFi.

La carpeta `Ingesta Python/` preserva el trabajo previo para trazabilidad y comparación. No forma parte del camino nominal de la nueva arquitectura. La carpeta `nifi/` contiene los elementos necesarios para reconstruir el flujo, parámetros no sensibles y documentación.

7.2 Inicialización del entorno

El flujo de arranque parte de Docker Compose. PostgreSQL debe superar su *health-check* antes de habilitar componentes que dependan de JDBC. NiFi requiere volúmenes persistentes para los repositorios que se decida conservar. Superset mantiene su proceso de inicialización y Redis se inicia como dependencia auxiliar.

La secuencia de entrada sigue siendo deliberadamente sencilla:

```
1 cp .env.example .env
2 docker compose up -d
3 docker compose ps
```

Listing 7.2: Secuencia conceptual de inicialización.

El repositorio ofrece comandos de conveniencia, como `make start`, `make demo`, `make test` y `make logs`, una vez sincronizados con la implementación final. Estos comandos no duplican la lógica de NiFi; actúan como puntos de entrada para un usuario que no necesita conocer todos los detalles del Compose.

7.3 Configuración de NiFi

La configuración se divide en valores del contenedor y parámetros del flujo. Las variables de infraestructura incluyen puertos, memoria, credenciales iniciales y rutas de persistencia. Los *Parameter Contexts* contienen valores funcionales como fecha inicial, fecha final, modo del pipeline, rutas Bronze y timeouts.

Los grupos de procesos consumen estos parámetros mediante la sintaxis propia de NiFi. Los valores sensibles se marcan como tales o se obtienen desde el entorno sin almacenarlos en el repositorio.

7.4 Descubrimiento de ficheros TLC

El grupo `10_TLC_DISCOVERY` determina qué activos deberían existir para cada combinación de servicio, año y mes. El resultado se representa mediante FlowFiles cuyos atributos contienen al menos servicio, año, mes, URL esperada y ruta objetivo.

Cada activo dispone de una clave lógica:

$$K_f = (s, y, m), \quad (7.1)$$

donde s es el servicio, y el año y m el mes. Antes de iniciar una descarga, NiFi consulta `control.ingestion_files`. Un activo finalizado correctamente puede enrutarse a `SKIPPED`; uno nuevo se registra como `DISCOVERED`.

Separar descubrimiento y descarga hace posible detectar activos ausentes, contabilizar trabajo pendiente y reanudar una ejecución sin repetir el catálogo completo de forma opaca.

7.5 Descarga mediante InvokeHTTP

La descarga se realiza mediante procesadores HTTP configurables. InvokeHTTP permite definir URL, método, timeout y gestionar distintas relaciones según el resultado de la petición [3]. La URL se construye a partir de los atributos y parámetros del FlowFile, evitando mantener un flujo distinto para cada mes.

La respuesta se valida antes de publicar el fichero en Bronze. Los códigos de éxito continúan hacia validación; errores transitorios se enrutan a retry; respuestas que indican que el recurso no existe se registran de forma diferenciada. La arquitectura no permite reintentos infinitos.

Cuando resulta viable se registra tamaño, checksum, código HTTP, intento y timestamps. El objetivo es que un fichero descargado pueda vincularse a una entrada de control y a sus eventos de procedencia.

7.6 Validación y publicación en Bronze

La validación previa a Silver comprueba propiedades físicas y estructurales:

1. respuesta HTTP satisfactoria;
2. contenido no vacío;
3. extensión y nombre esperados;
4. legibilidad como Parquet mediante el mecanismo implementado;
5. columnas mínimas para la familia y versión;
6. coherencia entre servicio, año, mes y destino;
7. ausencia de una versión ya validada con la misma identidad.

El fichero no se considera válido únicamente por haber sido transferido. El flujo mantiene un estado intermedio hasta completar las validaciones críticas y solo entonces publica el activo en la ruta Bronze definitiva y actualiza PostgreSQL.

En aquellos controles que requieran una librería no disponible directamente como procesador, NiFi puede invocar una utilidad encapsulada y versionada. Esta posibilidad se limita a validaciones concretas; la arquitectura evita reconstruir un segundo orquestador dentro de scripts auxiliares.

7.7 Rutas de éxito, fallo y cuarentena

Los FlowFiles se enrutan mediante relaciones y reglas explícitas. Procesadores como RouteOnAttribute permiten tomar decisiones basadas en atributos del flujo [6]. Conceptualmente se distinguen:

success el activo puede avanzar;

retry el fallo se considera transitorio;

failure se ha producido un error operativo no recuperado;

quarantine el contenido existe pero incumple un contrato crítico;

skipped el activo ya estaba procesado o no requiere trabajo.

Esta clasificación se refleja también en `control.ingestion_files`, de forma que el estado visual de NiFi y la auditoría histórica de PostgreSQL sean coherentes.

7.8 Conexión con PostgreSQL

NiFi utiliza un *Controller Service* de conexión para centralizar URL JDBC, driver y configuración del pool [1]. Las operaciones de lectura de estado pueden realizarse mediante procesadores SQL, mientras que las actualizaciones e inserciones se ejecutan mediante componentes de escritura de registros o SQL parametrizado [4].

La conexión se reutiliza entre grupos de procesos. No se crean credenciales distintas en cada procesador ni se codifican contraseñas en propiedades versionadas.

7.9 Carga hacia el área relacional

Bronze continúa siendo Parquet. Para construir Silver, la transición hacia PostgreSQL se realiza por particiones o lotes, evitando cargar el histórico completo en memoria. La estrategia exacta de lectura Parquet y carga se sincronizará con la implementación final y se medirá sobre la máquina objetivo.

NiFi coordina esta fase y registra su estado, pero las transformaciones semánticas permanecen en dbt. Una reejecución del mismo mes no debe añadir una segunda copia: la publicación relacional utiliza claves, particiones o estrategias de reemplazo explícitas.

7.10 Construcción de Silver con dbt

Los modelos Silver convierten cada familia en una representación analítica coherente. Se mantienen modelos especializados por servicio y posteriormente una proyección común de campos compatibles.

Un modelo conceptual puede seguir el patrón:

```

1 select
2     'yellow' as service_type ,
3     pickup_datetime ,
4     dropoff_datetime ,
5     pulocationid as pickup_location_id ,
6     dolocationid as dropoff_location_id ,
7     trip_distance ,
8     extract(epoch from (dropoff_datetime - pickup_datetime))/60.0
9     as trip_duration_minutes ,
10    total_amount ,
11    source_file
12 from {{ ref('stg_yellow_trips') }}
13 where pickup_datetime is not null

```

Listing 7.3: Esquema conceptual simplificado de una transformación Silver.

NiFi desencadena la ejecución cuando la entrada requerida se encuentra disponible. El SQL definitivo incorpora reglas de calidad, columnas históricas y documentación de cada campo.

7.11 Tests de dbt y promoción

Los tests genéricos incluyen `not_null`, `unique`, relaciones con dimensiones y valores aceptados. Se añaden tests singulares para reglas de negocio específicas. La ejecución de `dbt build` construye modelos y pruebas respetando dependencias.

El resultado de `dbt` se comunica al flujo de control. Un fallo crítico impide continuar hacia las fases que dependen del modelo Gold válido. De este modo, NiFi controla la secuencia, pero `dbt` conserva autoridad sobre sus transformaciones y tests.

7.12 Construcción de Gold

Gold se construye después de Silver y de sus controles. Las dimensiones de fecha, hora y zona se materializan de forma estable; los hechos derivan claves y métricas; y los marts agregan el conjunto mínimo necesario para dashboards.

`fact_zone_hourly_demand` completa el dominio zona-hora para incluir ceros reales. Si solo se agregaran los viajes existentes, desaparecerían las horas sin demanda y el modelo predictivo interpretaría un problema de conteo truncado.

7.13 Fuentes externas

Las fuentes externas se implementan en grupos separados y convergen en dimensiones o tablas de referencia. La meteorología, calendario, festivos y geografía presentan cadencias diferentes de los viajes TLC. NiFi permite asignar planificaciones distintas a cada proceso sin desplegar schedulers adicionales [7].

Los activos estáticos o de actualización poco frecuente no se descargan de nuevo en cada ejecución. El mismo principio de manifest y estado incremental se aplica a cada fuente.

7.14 Control de ejecuciones

Cada ejecución obtiene un `run_id`. `pipeline_runs` registra modo, rango temporal, servicios seleccionados, revisión Git y, cuando sea posible, versión del flujo. Las tareas relevantes escriben estados `PENDING`, `RUNNING`, `SUCCESS`, `FAILED`, `SKIPPED` o equivalentes.

El estado global no se reduce a un booleano. Una fuente externa opcional puede fallar sin invalidar necesariamente todos los datos TLC, mientras que un fallo en una transformación crítica de Silver sí debe bloquear etapas posteriores.

7.15 Data Provenance

Además del esquema de control, NiFi conserva información de procedencia sobre el recorrido de los FlowFiles. La interfaz de Data Provenance permite buscar eventos, inspeccionar detalles y visualizar el linaje del objeto a través del flujo [7].

Para la evaluación final se seleccionarán varios activos de demostración y se documentará su recorrido, por ejemplo:

```
1 DISCOVER
2   -> CHECK_MANIFEST
3   -> INVOKE_HTTP
4   -> VALIDATE
5   -> WRITE_BRONZE
6   -> REGISTER_METADATA
7   -> TRIGGER_DBT
```

Listing 7.4: Recorrido conceptual de un activo TLC en NiFi.

Esta evidencia complementa los logs y aporta una forma visual de demostrar trazabilidad.

7.16 Planificación

Los procesadores de entrada se configuran mediante planificación *Timer Driven* o *CRON Driven*, según la cadencia de la fuente [7]. La descarga mensual de TLC, la meteorología y los activos estáticos pueden tener planificaciones independientes.

El proyecto evita un contenedor scheduler separado. La planificación forma parte de la configuración del flujo y queda documentada junto con los Process Groups.

7.17 Activación de machine learning

El entrenamiento y la inferencia no se implementan como transformaciones visuales de NiFi. El grupo correspondiente activa el componente Python una vez que los features Gold requeridos están listos. Python se encarga de validación temporal, entrenamiento, métricas y persistencia de artefactos.

El resultado se escribe en `fact_model_predictions` y `fact_model_metrics` o en las estructuras definitivas equivalentes. NiFi registra el estado de la ejecución, pero no reemplaza el código científico.

7.18 Modo demo

El modo demo procesa un subconjunto temporal pequeño y utiliza la misma topología de NiFi que el modo completo. La diferencia se expresa mediante parámetros. El criterio de aceptación es que un entorno limpio genere:

- activos Bronze válidos;
- estado de ingesta registrado;
- tablas Silver y Gold;

- tests ejecutados;
- predicciones de ejemplo cuando el perfil demo active ML;
- datasets consultables desde Superset;
- eventos de procedencia consultables en NiFi.

7.19 Modo full

El modo completo utiliza el periodo 2018–2025 y los servicios configurados. El pipeline debe poder reanudarse sin volver a descargar meses correctamente validados.

La duración final será pendiente. Esta cifra se medirá sobre una revisión concreta del repositorio y se desglosará por fases en el capítulo de resultados.

7.20 Pruebas automatizadas

La estrategia de pruebas combina:

1. validación del Docker Compose y healthchecks;
2. comprobación de disponibilidad de NiFi y de los grupos de procesos esperados;
3. tests de idempotencia de la ingesta sobre una muestra;
4. rutas de fallo, retry y cuarentena;
5. tests de dbt sobre Silver y Gold;
6. tests Python del componente de machine learning;
7. una prueba end-to-end del modo demo.

La prueba end-to-end es el criterio principal de reproducibilidad. El resultado debe demostrar conexión real entre NiFi, Bronze, PostgreSQL, dbt, ML y Superset.

7.21 Rendimiento y uso de recursos

La introducción de NiFi incorpora un proceso Java persistente y repositorios internos, por lo que el consumo de memoria y disco se incluirá en la evaluación. La concurrencia de procesadores se dimensiona de forma conservadora para un equipo de 32 GB y se evita ejecutar simultáneamente tareas de ingesta masiva y entrenamientos especialmente costosos.

Las optimizaciones del plano analítico siguen basándose en lectura columnar, procesamiento por particiones, cargas por lotes, índices justificados, materializaciones incrementales y marts preagregados.

7.22 Documentación operativa

El repositorio incorpora instrucciones para inicialización, acceso a NiFi, restauración o importación del flujo, configuración de Parameter Contexts, ejecución de pruebas, reconstrucción de Superset y resolución de incidencias frecuentes. La documentación operativa forma parte del producto: una plataforma que solo puede ejecutar su autor no cumple el objetivo de reproducibilidad.

Bibliografía

- [1] Apache Software Foundation. Apache nifi dbcpconnectionpool controller service.
- [2] Apache Software Foundation. Apache nifi documentation: Components.
- [3] Apache Software Foundation. Apache nifi invokehttp processor.
- [4] Apache Software Foundation. Apache nifi putdatabaserecord processor.
- [5] Apache Software Foundation. Apache nifi registry and flow registry clients.
- [6] Apache Software Foundation. Apache nifi routeonattribute processor.
- [7] Apache Software Foundation. Apache nifi user guide.
- [8] Apache Software Foundation. Apache parquet documentation.
- [9] Apache Software Foundation. Apache superset documentation.
- [10] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [11] Dun Cao, Kai Zeng, Jin Wang, Pradip Kumar Sharma, Xiaomin Ma, Yonghe Liu, and Siyuan Zhou. Bert-based deep spatial-temporal network for taxi demand prediction. *IEEE Transactions on Intelligent Transportation Systems*, 23(7):9442–9454, 2022.
- [12] D. Correa and C. Moyano. Analysis and prediction of new york city taxi and uber demands. *Journal of Applied Research and Technology*, 21(5):886–898, 2023.
- [13] Databricks. What is the medallion lakehouse architecture?
- [14] dbt Labs. dbt developer hub.
- [15] Docker, Inc. Docker compose documentation.
- [16] Sabihah Sadat Faghih, Abolfazl Safikhani, Bahman Moghimi, and Camille Kamga. Predicting short-term uber demand in new york city using spatiotemporal modeling. *Journal of Computing in Civil Engineering*, 33(3), 2019.
- [17] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [18] Git Project. Git reference documentation.
- [19] Great Expectations. Great expectations documentation.
- [20] Hartwig H. Hochmair. Spatiotemporal pattern analysis of taxi trips in new york city. *Transportation Research Record*, 2542(1), 2016.
- [21] William H. Inmon. *Building the Data Warehouse*. John Wiley & Sons, 4 edition, 2005.

- [22] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.
- [23] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. John Wiley & Sons, 3 edition, 2013.
- [24] LightGBM Developers. Lightgbm documentation.
- [25] Huimin Luo, Jianming Cai, Kunpeng Zhang, Ruihang Xie, and Liang Zheng. A multi-task deep learning model for short-term taxi demand forecasting considering spatiotemporal dependences. *Journal of Traffic and Transportation Engineering (English Edition)*, 8(1):83–94, 2021.
- [26] Manik Mondal, Kazushi Sano, Teppei Kato, Chonnipa Puppateravanit, and Tomonori Watari. Predicting taxi demand in urban areas by the application of hybrid machine learning algorithm. *Journal of the Eastern Asia Society for Transportation Studies*, 15:1009–1028, 2024.
- [27] National Centers for Environmental Information. Climate data online and historical climate data.
- [28] New York City Taxi and Limousine Commission. Aggregated reports and tlc data.
- [29] New York City Taxi and Limousine Commission. Tlc trip record data. Portal oficial de datos de viajes.
- [30] New York City Taxi and Limousine Commission. Tlc trip records user guide.
- [31] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [32] PostgreSQL Global Development Group. Postgresql documentation.
- [33] A. P. Riascos and José L. Mateos. Networks and long-range mobility in cities: A study of more than one billion taxi trips in new york city. *Scientific Reports*, 10:4022, 2020.
- [34] Abolfazl Safikhani, Camille Kamga, Sandeep Mudigonda, Sabiheh Sadat Faghieh, and Bahman Moghimi. Spatio-temporal modeling of yellow taxi demands in new york city using generalized star models. *International Journal of Forecasting*, 36(3):1138–1148, 2020.
- [35] scikit-learn developers. Poissonregressor.
- [36] scikit-learn developers. Randomforestregressor.
- [37] Patrick Toman, Jingyue Zhang, Nalini Ravishanker, and Karthik Konduri. Spatio-temporal analysis of ridesourcing and taxi demand by taxi zones in new york city. *arXiv preprint arXiv:2008.00568*, 2020.
- [38] Panos Vassiliadis. A survey of extract–transform–load technology. *International Journal of Data Warehousing and Mining*, 5(3):1–27, 2009.
- [39] Yanan Zhang, Xueliang Sui, and Shen Zhang. Exploring spatio-temporal impact of covid-19 on citywide taxi demand: A case study of new york city. *PLOS ONE*, 19(4):e0299093, 2024.